

Computer System Architecture

Lesson Study Guide: Software Types & Computer Architectures

Course: ICT 1022 (2.0)

Institution: University of Sri Jayewardenepura

Department: Computer Science

1. Software Classification

A **program** or **software** consists of a set of step-by-step instructions executed by a computer to solve a specific problem. Software is broadly classified into two primary categories:

Application Software

USER-FOCUSED

Programs designed to help users solve specific, real-world tasks or user-level problems. They usually reside on local storage (HDD, SSD, USB) and run on top of system software.

Examples:

- Financial accounting packages (e.g., Tally)
- Mathematical software (MATLAB, Mathematica)
- Everyday applications (Cab booking, health monitors)

System Software

HARDWARE MANAGEMENT

A collection of programs that manages system resources, handles hardware operations, and enables user applications to run seamlessly.

Examples:

- Operating Systems (Windows, Linux, macOS)
- Compilers & Assemblers
- Linkers & Loaders
- Editors & Debuggers

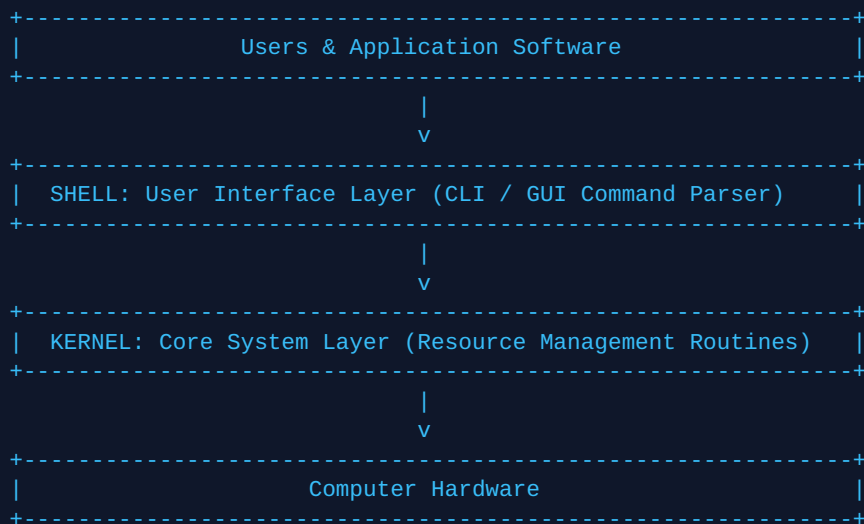
Key Responsibilities of System Software

- **Resource & File Management:** Organizing data in secondary storage devices and allocating memory.
- **Input/Output (I/O) Control:** Managing peripherals such as keyboards, displays, and storage drives.
- **Program Translation:** Converting human-readable source code into binary object form.
- **Program Execution:** Linking object modules and loading programs into main memory for execution.

2. Operating System (OS) Architecture

An **Operating System** is a foundational system software instance that runs continuously from system startup until power down. It acts as an interface between computer hardware and user application software.

Layered OS Architecture



- **Kernel:** The inner core of the OS containing low-level routines for CPU scheduling, memory management, and device interaction.
- **Shell:** The outermost layer that provides an interface (Command Line or GUI) for users to request services from the kernel.

Operating System Design Goals by Application

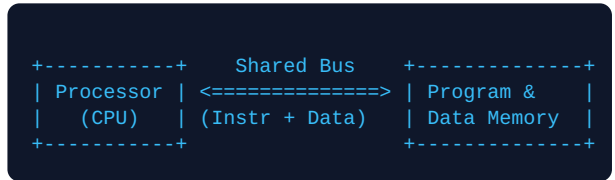
OS Type	Primary Design Objective	Key Operating Principles
Classical Multi-programming	Maximize Resource Utilization	Keeps multiple programs loaded in RAM; switches to another task whenever the active program is blocked by I/O.
Modern Time-Sharing	Minimize User Response Time	Shares CPU time rapidly across multiple interactive users, giving each user the illusion of exclusive access.
Real-Time Systems (RTOS)	Guarantee Strict Deadlines	Interrupt-driven operation; immediately pauses ongoing tasks when urgent real-world tasks arrive (e.g., missile control, medical equipment).
Mobile Systems	User Responsiveness & Efficiency	Prioritizes foreground responsiveness; aggressively terminates or suspends memory-heavy background apps to conserve battery/RAM.

3. Fundamental Computer Architectures

While calculators and computers share similar arithmetic circuitry, computers are distinguished by their ability to store and execute variable instruction sequences autonomously from memory.

Von-Neumann Architecture

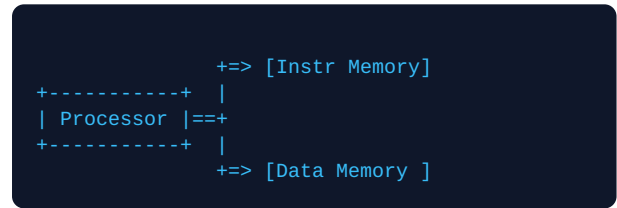
Uses a **single shared memory space** and a unified bus to store and transfer both instructions and data.



- **Pros:** Flexible, simpler circuitry, lower hardware cost.
- **Cons:** *Von-Neumann Bottleneck* — CPU cannot read instructions and access data simultaneously.

Harvard Architecture

Uses **physically separate memory units** and distinct buses for program instructions and data.



- **Pros:** Parallel instruction and data access; avoids bus contention.
- **Cons:** Increased wiring complexity and higher hardware cost.

4. Instruction Pipelining & Memory Conflicts

Instruction execution in modern CPUs is divided into 5 distinct stages to increase throughput by overlapping execution cycles:

5-Stage Instruction Pipeline

1. **IF (Instruction Fetch):** Retrieves the instruction from memory.
2. **ID (Instruction Decode):** Decodes the instruction and reads register contents.
3. **EX (Execute):** Performs operations using the Arithmetic Logic Unit (ALU).
4. **MEM (Memory Access):** Reads from or writes to data memory (if needed).
5. **WB (Write Back):** Writes the result back to the register file.

Pipeline Execution Timing Diagram

Instruction	Cycle 1	Cycle 2	Cycle 3	Cycle 4	Cycle 5	Cycle 6	Cycle 7
Instr 1	IF	ID	EX	MEM	WB	-	-
Instr 2	-	IF	ID	EX	MEM	WB	-
Instr 3	-	-	IF	ID	EX	MEM	WB
Instr 4	-	-	-	IF	ID	EX	MEM

Conflict Analysis (Cycle 4): Notice that in Cycle 4, *Instr 4* requires an **Instruction Fetch (IF)** while *Instr 1* performs a **Memory Access (MEM)**.

- Under **Von-Neumann Architecture**, both operations contend for the single shared bus, causing a pipeline stall.
- Under **Harvard Architecture**, IF reads from Instruction Memory while MEM accesses Data Memory concurrently over separate buses without speed penalty.

5. Emerging Architectures: In-Memory Computing

To overcome the physical speed limitations and energy losses associated with transferring data between CPU and main memory, researchers are developing **In-Memory Computing Architectures**.

Memristors (Memory Resistors)

- **Concept:** Nanoscale circuit elements that retain their electrical resistance state even when powered off.
- **Function:** Enable non-volatile, high-capacity resistive memory (ReRAM) that can perform logical and computational operations directly within the memory arrays.
- **Impact:** Eliminates bus transfer bottlenecks completely, dramatically reducing power consumption and execution latency for modern AI and data-intensive computing.